

Operating Systems

Complete Study Notes

Unit I & Unit II

Unit I: Introduction
Unit II: Processes and Threads

Reference:

Operating System Concepts
Silberschatz, Gagne & Galvin

Exam-Oriented Study Material

August 12, 2026

Contents

1	UNIT I – INTRODUCTION	2
1.1	What is an Operating System?	2
1.2	Operating System Operations	2
1.3	Resource Management	3
1.4	Operating System Services	4
1.5	Virtualization	4
1.6	User and Operating System Interface	5
1.7	System Calls	5
1.8	Operating System Structures	6
1.9	Building and Booting an Operating System	7
2	Unit I – Possible University Exam Questions	8
2.1	Part A – 2/3 Marks	8
2.2	Part B – 10/13/16 Marks	8
3	UNIT II – PROCESSES AND THREADS	9
3.1	Process Concept	9
3.2	Process Control Block	9
3.3	Process Scheduling	10
3.4	Operations on Processes	11
3.5	Interprocess Communication	11
3.6	Shared-Memory IPC	12
3.7	Message-Passing IPC	12
3.8	Examples of IPC Systems	13
3.9	Threads	14
3.10	Multithreading Models	15
3.11	Pthreads	15
4	Unit II – Possible University Exam Questions	16
4.1	Part A – 2/3 Marks	16
4.2	Part B – 10/13/16 Marks	17
5	Quick Revision Table	18
6	Exam Preparation Tips	18

1 UNIT I – INTRODUCTION

1.1 What is an Operating System?

An Operating System (OS) is **system software that acts as an intermediary between the user/application programs and the computer hardware**. It manages hardware resources and provides a convenient and efficient environment for program execution.

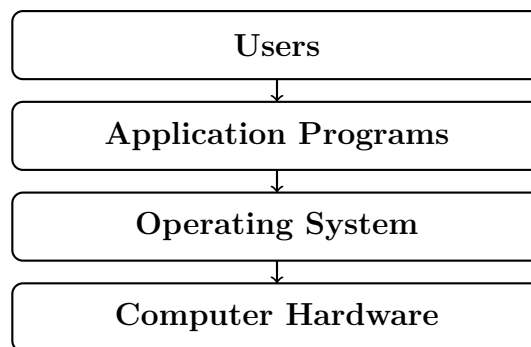
Two Goals of an Operating System

1. **Convenience** – makes the computer system easy to use.
2. **Efficiency** – ensures computer hardware resources are used efficiently.

Two Views of an Operating System

- **User view** – OS makes the system convenient and user-friendly through interface, performance, and ease of use.
- **System view** – OS acts as a **resource allocator** and a **control program** that manages hardware and prevents errors or improper use.

Layered Structure of a Computer System



1.2 Operating System Operations

Modern operating systems are **interrupt driven**. Events are signaled by hardware interrupts or software-generated traps/exceptions.

- **Interrupt Vector**: contains addresses of interrupt service routines indexed by interrupt numbers.
- **Trap (Exception)**: software-generated interrupt caused by an error or a user request such as a system call.

Dual-Mode / Multimode Operation

To protect hardware from misuse, operating systems provide at least two modes.

- **User mode**: restricted execution; non-privileged instructions.
- **Kernel mode**: privileged execution; privileged instructions are allowed.

The **mode bit** in hardware distinguishes between user and kernel execution modes.

Privileged Instructions

Examples include:

- I/O control
- Timer management
- Interrupt management

These instructions can be executed only in kernel mode.

Timer

A timer prevents a user program from monopolizing the CPU or entering an infinite loop.

The timer generates an interrupt after a specified period, transferring control back to the operating system.

1.3 Resource Management

The operating system manages several important resources.

Process Management

A **process** is a program in execution.

OS responsibilities include:

- Creating and deleting processes
- Suspending and resuming processes
- Process synchronization
- Interprocess communication
- Deadlock handling

Memory Management

The OS:

- Tracks which parts of memory are in use.
- Allocates and deallocates memory.
- Decides which processes/data move into and out of memory.

File Management

Responsibilities include:

- File creation and deletion
- Directory management
- Mapping files onto secondary storage
- Backup

Mass Storage and I/O Management

Includes:

- Free-space management
- Storage allocation
- Disk scheduling
- Device drivers
- Buffering
- Caching
- Spooling

Cache Management

Information is normally kept in a faster storage system as a cache for larger and slower storage systems.

1.4 Operating System Services

Services for Users

1. **User Interface** – CLI, GUI, batch, touchscreen.
2. **Program Execution** – load, execute, and terminate programs.
3. **I/O Operations** – provide I/O facilities to running programs.
4. **File-System Manipulation** – create, delete, read, and write files/directories.
5. **Communication** – communication between processes.
6. **Error Detection** – detects hardware, I/O, and program errors.

Services for Efficient System Operation

7. **Resource Allocation**
8. **Accounting**
9. **Protection and Security**

1.5 Virtualization

Virtualization allows multiple operating systems or execution environments to run on the same physical hardware.

A **VMM (Virtual Machine Manager)** or **Hypervisor** creates and manages virtual machines.

- **Host**: underlying hardware/system.
- **Guest**: operating system running inside a virtual machine.

Types of Hypervisors

1. **Type 0:** hardware/firmware-based.
2. **Type 1:** runs directly on hardware (bare-metal).
3. **Type 2:** runs on top of a host operating system.

Benefits

- Testing
- Running multiple operating systems
- Development
- Server consolidation

1.6 User and Operating System Interface

Command Line Interface

CLI or shell allows users to enter commands directly.

Examples include:

- Bash
- C shell

Graphical User Interface

GUI uses:

- Windows
- Icons
- Menus
- Pointer

This is commonly called the **WIMP** interface.

Touchscreen Interface

Used mainly in mobile devices and relies on gestures and touch interaction.

1.7 System Calls

A **system call** is a programming interface to the services provided by the operating system.

Applications normally access system calls through APIs such as:

- POSIX API
- Win32 API
- Java API

Parameter Passing

Parameters can be passed:

1. Through registers.
2. Through a block/table in memory.
3. Through the stack.

Types of System Calls

1. **Process Control**
2. **File Management**
3. **Device Management**
4. **Information Maintenance**
5. **Communications**
6. **Protection**

Example

```
1 printf() --> write()
```

Here, `printf()` is a library function and `write()` represents a system call.

1.8 Operating System Structures

Simple Structure – MS-DOS

Not divided into clearly separated modules or levels.

Monolithic Structure – Traditional UNIX

Consists mainly of:

- System programs
- Kernel

The kernel contains the system-call interface, CPU scheduling, memory management, file system, and I/O management.

Layered Approach

The OS is divided into layers, where each layer is built on top of lower layers.

Advantages:

- Modularity
- Easier debugging

Disadvantages:

- Potential performance overhead
- Difficult layer definition

Microkernels

A microkernel moves as much functionality as possible from the kernel into user space.

Communication between modules occurs through **message passing**.

Advantages:

- Easier extension
- Easier porting
- Improved reliability
- Improved security

Disadvantage: communication overhead.

Modules

Loadable kernel modules allow services to be dynamically linked into the kernel.

Examples include Linux, Solaris, and macOS.

Hybrid Systems

Hybrid systems combine several approaches.

Examples:

- Linux
- Windows
- macOS
- iOS

1.9 Building and Booting an Operating System

Building an OS

Typical steps:

1. Write/generate source code.
2. Compile.
3. Link with libraries and object files.
4. Create a bootable disk image.
5. Install on the computer.

SYSGEN obtains information about the hardware configuration and generates an OS tailored to that system.

Booting

The **bootstrap program** locates the operating-system kernel, loads it into memory, and starts its execution.

Common boot technologies include:

- BIOS
- UEFI
- GRUB

2 Unit I – Possible University Exam Questions

2.1 Part A – 2/3 Marks

1. Define Operating System.
2. What is a system call? Give an example.
3. What is dual-mode operation? Why is it needed?
4. Define virtualization and hypervisor.
5. What is a bootstrap program?
6. Differentiate BIOS and UEFI.
7. What is a trap?

2.2 Part B – 10/13/16 Marks

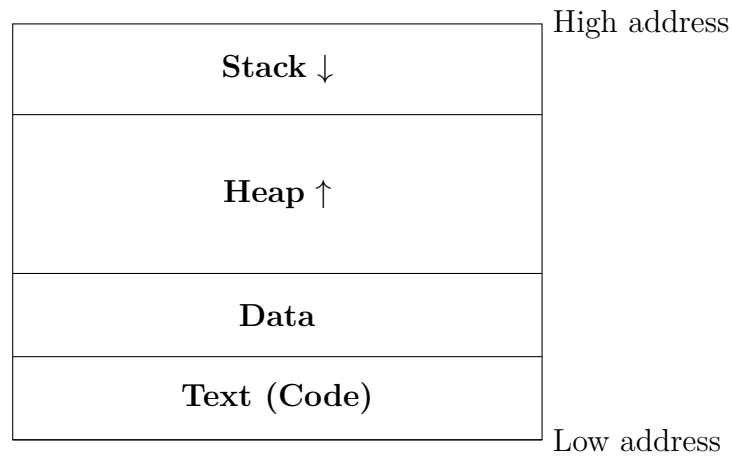
1. Explain the various operating system services with a neat diagram.
2. Explain the different types of system calls with examples.
3. Discuss different Operating System structures.
4. Explain dual-mode operation and resource protection.
5. Explain OS resource management functions.
6. Explain virtualization and types of hypervisors.
7. Explain building and booting an OS.
8. Differentiate CLI and GUI.

3 UNIT II – PROCESSES AND THREADS

3.1 Process Concept

A **process** is a program in execution. A program is passive, while a process is active.

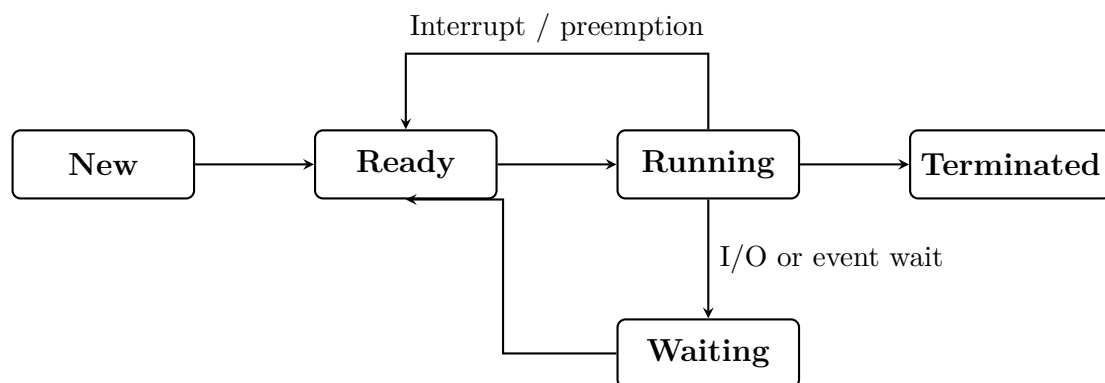
Process Memory Layout



Process States

1. New
2. Ready
3. Running
4. Waiting
5. Terminated

Process State Diagram



3.2 Process Control Block

Each process is represented by a **PCB**.

The PCB contains:

- Process state
- Program counter

- CPU registers
- CPU scheduling information
- Memory-management information
- Accounting information
- I/O status information

3.3 Process Scheduling

The main goal of process scheduling is to maximize CPU utilization and provide efficient time sharing.

Scheduling Queues

- **Job queue:** all processes in the system.
- **Ready queue:** processes ready for CPU execution.
- **Device queues:** processes waiting for I/O devices.

Schedulers

1. Long-term scheduler

- Selects processes from disk/job pool.
- Loads them into memory.
- Controls degree of multiprogramming.

2. Short-term scheduler

- Selects a process from the ready queue.
- Allocates CPU.
- Must be very fast.

3. Medium-term scheduler

- Swaps processes out of memory.
- Later swaps them back in.

Process Types

- **I/O-bound:** many short CPU bursts.
- **CPU-bound:** fewer, longer CPU bursts.

Context Switch

During a context switch:

1. Save the state of the old process into its PCB.
2. Load the state of the new process from its PCB.

Context-switching time is pure overhead.

3.4 Operations on Processes

Process Creation

A parent process can create child processes using system calls such as `fork()` in UNIX.

```
1 pid_t pid = fork();
2
3 if (pid == 0) {
4     // Child process
5 } else {
6     // Parent process
7 }
```

`fork()`, `exec()`, `wait()`, `exit()`

- `fork()` – creates a child process.
- `exec()` – replaces the process's memory with a new program.
- `wait()` – parent waits for child.
- `exit()` – terminates a process.

Zombie Process

A process that has terminated but whose parent has not yet called `wait()`.

Orphan Process

A process whose parent terminates before the child.

3.5 Interprocess Communication

Processes may be:

- **Independent**
- **Cooperating**

Cooperating processes require IPC.

Reasons for Cooperating Processes

- Information sharing
- Computation speedup
- Modularity
- Convenience

Two Fundamental IPC Models

1. Shared Memory
2. Message Passing

3.6 Shared-Memory IPC

In shared-memory IPC, processes communicate by reading and writing a shared memory region.

It is generally faster after setup because processes do not need kernel intervention for every data exchange.

Producer–Consumer Problem

The producer generates data while the consumer consumes it.

A buffer is used between them.

- **Unbounded buffer:** no practical size limit.
- **Bounded buffer:** fixed size.

```
1 #define BUFFER_SIZE 10
2
3 typedef struct {
4     /* item data */
5 } item;
6
7 item buffer[BUFFER_SIZE];
8
9 int in = 0;
10 int out = 0;
11
12 // in == out => buffer empty
13 // (in + 1) % BUFFER_SIZE == out => buffer full
```

3.7 Message-Passing IPC

Message passing allows processes to communicate without sharing the same address space.

Basic operations:

```
1 send(message)
2 receive(message)
```

Naming

Direct Communication Processes explicitly identify each other.

```
1 send(P, message)
2 receive(Q, message)
```

Indirect Communication Messages are sent through mailboxes or ports.

```
1 send(A, message)
2 receive(A, message)
```

Synchronization

- **Blocking:** sender/receiver waits.
- **Non-blocking:** sender/receiver continues execution.

Buffering

- **Zero capacity:** sender waits for receiver; rendezvous.
- **Bounded capacity:** finite queue.
- **Unbounded capacity:** sender does not wait due to queue capacity.

3.8 Examples of IPC Systems

POSIX Shared Memory

Common functions include:

- `shmget()`
- `shmat()`
- `shmdt()`
- `shmctl()`

Mach

Mach uses message passing extensively and provides communication through ports.

The `mach_msg()` function supports message communication.

Windows ALPC

Windows uses **Advanced Local Procedure Calls (ALPC)** for communication between local client and server processes.

Pipes

Ordinary Pipes

- Unidirectional
- Producer-consumer communication
- Commonly used between related processes

Named Pipes

- More flexible
- Can be bidirectional
- Do not require a parent-child relationship
- Can be used by multiple processes

3.9 Threads

A **thread** is the basic unit of CPU utilization.

A thread contains:

- Thread ID
- Program counter
- Register set
- Stack

Threads belonging to the same process share:

- Code
- Data
- Open files
- Other OS resources

Benefits of Multithreading

1. **Responsiveness**
2. **Resource sharing**
3. **Economy**
4. **Scalability**

Concurrency vs Parallelism

- **Concurrency:** multiple tasks make progress during overlapping periods.
- **Parallelism:** multiple tasks execute simultaneously on multiple processing units.

Amdahl's Law

$$\text{Speedup} \leq \frac{1}{S + \frac{1-S}{N}}$$

where:

- S = serial portion of the program.
- N = number of processing cores.

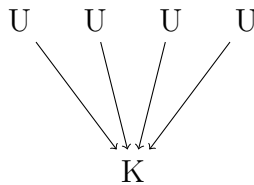
Types of Parallelism

- **Data parallelism:** same operation on different portions of data.
- **Task parallelism:** different tasks executed by different threads.

3.10 Multithreading Models

Many-to-One

Many user threads map to one kernel thread.

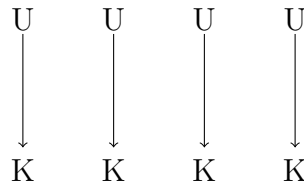


Drawbacks:

- Blocking system call can block entire process.
- No multicore parallelism.

One-to-One

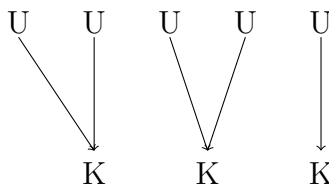
Each user thread maps to a kernel thread.



Provides greater concurrency but has greater overhead.
Examples include Linux and Windows.

Many-to-Many

Many user threads are multiplexed over multiple kernel threads.



Two-Level Model

A variation of many-to-many where a user thread may be bound to a particular kernel thread.

3.11 Pthreads

Pthreads is the POSIX standard API for thread creation and synchronization.

Important functions:

- `pthread_create()`
- `pthread_join()`
- `pthread_attr_init()`
- `pthread_exit()`

Pthread Sum Program

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int sum;
6
7 void *runner(void *param)
8 {
9     int upper = atoi((char *)param);
10
11     sum = 0;
12
13     for (int i = 1; i <= upper; i++)
14         sum += i;
15
16     pthread_exit(0);
17 }
18
19 int main(int argc, char *argv[])
20 {
21     pthread_t tid;
22     pthread_attr_t attr;
23
24     pthread_attr_init(&attr);
25
26     pthread_create(
27         &tid,
28         &attr,
29         runner,
30         argv[1]
31     );
32
33     pthread_join(tid, NULL);
34
35     printf("sum = %d\n", sum);
36
37     return 0;
38 }
```

Pthread Scope

- PTHREAD_SCOPE_SYSTEM – one-to-one.
- PTHREAD_SCOPE_PROCESS – many-to-many.

4 Unit II – Possible University Exam Questions

4.1 Part A – 2/3 Marks

1. Define process. Draw the process state diagram.
2. What is a PCB? List its contents.

3. Differentiate process and thread.
4. What is a context switch?
5. Define zombie and orphan process.
6. What is the difference between direct and indirect communication?
7. Define Amdahl's Law.
8. What are Pthreads?

4.2 Part B – 10/13/16 Marks

1. Explain the process state diagram and PCB.
2. Explain process scheduling and the different schedulers.
3. Discuss process creation and termination using UNIX system calls.
4. Explain shared memory and message passing.
5. Explain the Producer–Consumer problem.
6. Explain message passing: naming, synchronization, and buffering.
7. Explain any two IPC systems.
8. Explain multithreading models.
9. Explain multithreading benefits and Amdahl's Law.
10. Write and explain a Pthreads program to compute the sum from 1 to N .

5 Quick Revision Table

Topic	Key Term	One-liner
OS	Resource allocator/-control program	Manages hardware and software resources
Dual mode	Mode bit	Kernel mode and user mode provide protection
Virtualization	VMM / Hypervisor	Abstracts hardware to run multiple OS environments
System call	Trap into kernel	Interface to OS services
OS Structure	Monolithic / Layered / Microkernel / Modules / Hybrid	Design of kernel organization
Process	PCB	Program in execution together with its state
Scheduler	Long / Short / Medium term	Selects processes for memory or CPU
IPC	Shared memory / Message passing	Communication between processes
Producer–Consumer	Bounded buffer	Classic shared-memory synchronization problem
Thread	Lightweight execution unit	Shares code/data while maintaining its own stack and registers
Multithreading Model	Many-to-One / One-to-One / Many-to-Many	Maps user threads to kernel threads
Pthreads	POSIX standard	API for thread creation and synchronization

6 Exam Preparation Tips

Practice drawing the following diagrams by hand:

1. Process state diagram
2. Process memory layout
3. Process scheduling queue diagram
4. Multithreading model diagrams
5. IPC diagrams

These diagrams are commonly asked in university examinations and can improve answer presentation.

Important Things to Memorize

- `fork()` / `exec()` / `wait()` / `exit()`
- Pthreads sum program
- Amdahl's Law
- Process states
- PCB contents
- Types of IPC
- Multithreading models